Access Control Automation with Python

Course: Google Cybersecurity Professional Certificate
Project Type: Python Security Automation
Environment: Simulated Healthcare Organization

1. Project Description

In this project, I worked through a Python lab to update a file containing IP addresses authorized to access restricted information.

The algorithm compares an existing IP address allow list against a list of addresses that should no longer have access. It removes matching addresses and updates the original file.

This project demonstrates how Python can automate file management and access-control-related tasks in cybersecurity.

2. Open the File Containing the Allow List

I used the `with` statement and the `open()` function to access the file containing the IP address allow list.

```
None
with open(import_file, "r") as file:

    ip_addresses = file.read()
```

The `"r"` mode opens the file for reading. The `with` statement automatically closes the file when the block finishes.

3. Read the File Contents

I used `.read()` to read the file and store its contents in the `ip_addresses` variable.

```
None
ip_addresses = file.read()
```

At this stage, `ip_addresses` contains the file contents as a string.

4. Convert the String into a List

I used `.split()` to convert the string of IP addresses into a list.

```
None
ip_addresses = ip_addresses.split()
```

This allows the algorithm to process each IP address as an individual list element.

5. Iterate Through the IP Address List

I used a `for` loop to examine the IP addresses in the allow list.

```
None
for element in ip_addresses:
```

During each iteration, `element` represents the current IP address being checked.

6. Remove IP Addresses That Are No Longer Authorized

I used an `if` statement to check whether the current IP address appeared in `remove_list`.

```
None
if element in remove_list:

    ip_addresses.remove(element)
```

If the address appears in `remove_list`, the algorithm removes it from the allow list.

This matches the original Coursera lab implementation.

7. Update the File with the Revised IP Address List

After removing the specified addresses, I used `.join()` to convert the list back into a string.

```
None
ip_addresses = " ".join(ip_addresses)
```

The remaining IP addresses are separated by spaces.

I then opened the original file in write mode.

```
None
with open(import_file, "w") as file:

    file.write(ip_addresses)
```

The `"w"` mode replaces the original file contents with the revised allow list.

8. Complete Algorithm

The following is the complete algorithm structure from the Coursera lab, combining the individual steps into a reusable function.

```
None
def update_file(import_file, remove_list):

    with open(import_file, "r") as file:

        ip_addresses = file.read()


    ip_addresses = ip_addresses.split()


    for element in ip_addresses:

        if element in remove_list:

            ip_addresses.remove(element)


    ip_addresses = " ".join(ip_addresses)


    with open(import_file, "w") as file:

        file.write(ip_addresses)
```

The function accepts the allow-list filename and the IP addresses that should be removed.

Implementation note: This reproduces the course approach. Removing elements while iterating over the same list can cause some addresses to be skipped. A safer implementation would iterate over `remove_list` or create a filtered list.

9. Skills Demonstrated

- Python Programming
- Security Automation
- Access Control
- File Input/Output
- String and List Manipulation
- Loops and Conditional Statements
- IP Address Allow List Management

10. Summary

This project provided hands-on experience using Python to automate updates to an IP address allow list.

I practiced file operations, strings, lists, loops, and conditional statements to identify and remove addresses that should no longer have access.

The exercise demonstrated how Python can automate repetitive access-control tasks and support security administration.

Completed as part of the Google Cybersecurity Professional Certificate in a simulated lab environment. The explanations are based on my original portfolio activity, and the consolidated function follows the course's reference implementation.